BattleOps Week 4 Assignments

MONDAY: PROMETHEUS + GRAFANA UNDER LOAD

SETUP

Deploy kube-prometheus-stack (if not already installed):

- helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
- helm repo update
- helm install kps prometheus-community/kube-prometheus-stack -n observability --create-namespace
- Verify: kubectl get pods -n observability
- Grafana port-forward: kubectl port-forward -n observability svc/kps-grafana 3000:80

LAB 1: BREAK PROMETHEUS WITH HIGH CARDINALITY

Step 1: Deploy high-cardinality metric generator

kubectl apply -f https://raw.githubusercontent.com/prometheus/client_golang/main/examples/random/metrics-server.yaml

This app exposes a label with a random value every scrape → **explodes Prometheus memory**.

Step 2: Watch Prometheus blow up

Check memory: kubectl top pod -n observability | grep prometheus
Check Prometheus status: curl -s localhost:9090/api/v1/status/tsdb | jq
Look for:

- numSeries
- memoryInBytes
- labelPairs

If numbers explode → high cardinality confirmed.

Step 3: FIX using relabel configs

Edit the Prometheus config: kubectl edit cm kps-kube-prometheus-stack-prometheus -n observability

Add: metric_relabel_configs: - source_labels: [random_label]   regex: ".*" action: drop

Apply: kubectl rollout restart deploy kps-kube-prometheus-stack-prometheus -n observability

WHY WE DO THIS

- High cardinality **kills Prometheus instantly**
- Every enterprise team faces this at scale
- Prometheus cannot handle **unique-per-user / unique-per-request** labels

TUESDAY: LOKI + FLUENTBIT: INGESTION FAILURE WAR-ROOM

SETUP

Install Loki + FluentBit via Helm: helm repo add grafana https://grafana.github.io/helm-charts
helm install loki grafana/loki-stack -n observability

LAB 2: CREATE A LOGGING STORM

Step 1: Deploy a log-bomb

kubectl run log-bomber --image=busybox -- sh -c \ "while true; do echo $(date) ERROR Bomb; done"

This generates **thousands of logs per second**.

Step 2: check FluentBit Resource Saturation

kubectl top pods -n observability | grep fluent

Signs of trouble:

- High CPU
- High memory
- Pod restarts
- Dropped logs

Step 3: Check Loki Ingest Metrics

kubectl port-forward -n observability svc/loki 3100:3100

curl -s "http://localhost:3100/metrics" | grep -E 'ingest|dropped'

Focus on:

- loki_ingester_lines_received_total
- loki_ingester_lines_dropped_total
- loki_request_duration_seconds_bucket

Step 4: Fix the Pipeline

Increase FluentBit buffer: kubectl edit cm loki-fluent-bit -n observability

Change: Mem_Buf_Limit 10MB → 100MB

Restart: kubectl rollout restart ds loki-fluent-bit -n observability

WHY WE DO THIS

This is exactly what happens in real outages:

- Logs spike
- Ingestion collapses
- Debugging becomes impossible

WEDNESDAY: DISTRIBUTED TRACING FAILURE MODES

SETUP

Install OpenTelemetry Collector + Tempo: helm install tempo grafana/tempo -n observability
helm install otel grafana/opentelemetry-collector -n observability

LAB 3: BREAK TRACING BY MAXING SAMPLING

Step 1: Edit OTel collector config

kubectl edit cm otel-opentelemetry-collector -n observability

Set sampling to 100%: sampler: type: always_on

Reload: kubectl rollout restart deploy otel-opentelemetry-collector -n observability

Expected outcome:

- High CPU
- Trace pipeline lag
- Missing spans

Step 2: Debug Slow Spans

Open Grafana → Tempo → Search → Filter for: duration > 200ms

Find:

- Slowest microservice
- Slowest RPC
- Missing parent/child spans

Step 3: FIX by enabling parent-based sampling

Sampler: type: parentbased_always_on

THURSDAY: AUTOSCALING FAILURE SIMULATION

SETUP

Deploy demo app: kubectl apply -f https://k8s.io/examples/application/php-apache.yaml

Create HPA: kubectl autoscale deployment php-apache --cpu-percent=50 --min=1 --max=10

LAB 4: BREAK THE HPA (THROTTLING INCIDENT)

Step 1: Make the pod impossible to scale

Edit limits so CPU is too low: kubectl edit deploy php-apache

Set: resources: limits: cpu: 50m

Prometheus will see “low usage,” so **HPA NEVER TRIGGERS**.

Step 2: Generate Load

hey -z 30s -q 50 http://<NODE-IP>:<PORT>/

Observe: kubectl top pods || kubectl describe hpa php-apache || kubectl describe pod <pod-name>

You’ll see:

- CPU throttling
- High latency
- HPA not scaling

Step 3: FIX Scaling

Set CPU limit properly: cpu: 500m
Then: kubectl rollout restart deploy php-apache

WHY WE DO THIS

Most HPA outages in production = **wrong CPU limits**.

FRIDAY: SLOs, BURN RATES & INCIDENT DRILL

SETUP

Deploy a fake app with error injection: kubectl apply -f https://raw.githubusercontent.com/istio/istio/master/samples/httpbin/httpbin.yaml

Inject faults: kubectl apply -f https://raw.githubusercontent.com/istio/istio/master/samples/httpbin/httpbin-delay.yaml

LAB 5: BURN RATE ALERTING & INCIDENT SIMULATION

Step 1: Create an SLO (99% availability)

Define: latency < 300ms || error rate < 1%

Step 2: Create Burn Rate Alerts

Short window (page-worthy): burn_rate_1h > 2%

Long window (ticket-worthy): burn_rate_24h > 5%

Apply rules: kubectl apply -f slo-rules.yaml

Step 3: Trigger the Incident

Send faulty traffic: hey -z 1m -q 20 http://<httpbin-url>/status/500
Watch:

- Alerts firing
- Error budget consumption
- p99 latency spike

Step 4: Write the SLO-based RCA

You will answer:

1. What was the user-visible impact?
2. Which SLO was violated?
3. What is the burn rate?
4. What root cause did metrics/logs/traces reveal?
5. What's the fix?
6. What's the prevention strategy?

THE DEVOPS WAR ROOM